



DSA 4060 Recommender Systems

Week 1 Practical

Build and Publish a Popularity Based Movie Recommender

In this practical, you will build a complete introductory recommender-system project using Python, Pandas, the MovieLens small dataset, and Jupyter Notebook or Google Colab. You will inspect user-item interactions, calculate movie popularity, produce a simple ranked recommendation list, test the output, document the project, and publish it to GitHub.

The recommender created this week is non-personalized: every user receives the same ranked list. It provides a transparent baseline that later personalized models can be compared against.

Learning Outcomes

1. Explain users, items, interactions, explicit feedback, and non-personalized recommendations.
2. Load, inspect, validate, and summarize a user-item ratings dataset.
3. Build popularity and weighted-rating recommendation baselines.
4. Interpret the strengths and limitations of the generated recommendations.
5. Organize a reproducible project and publish it to a GitHub repository.

Practical Overview

Item	Details
Recommended duration	Approximately 2.5 to 3 hours.
Mode	Individual practical.
Tools	Python 3, Jupyter Notebook or Google Colab, Pandas, Matplotlib, and GitHub.
Dataset	MovieLens latest-small: movies.csv and ratings.csv.
Main output	A Top 10 popularity-based movie recommender and a weighted-rating recommender.
Repository name	dsa4060-week1-recommender.
Submission	GitHub repository URL and required screenshots.

Academic and Data Use

Use the MovieLens dataset for learning and retain its accompanying README and licence information when distributing dataset files. Write your own explanations. Do not upload passwords, tokens, private information, or unrelated large files to GitHub.

Part 1 Understand the Recommendation Problem

1.1 Core Concepts

Concept	Meaning in this practical
User	A person represented by userId who rates movies.
Item	A movie represented by movieId.
Interaction	A rating given by a user to a movie.
Explicit feedback	A directly stated preference, such as a rating from 0.5 to 5.0.
Recommendation	A ranked list of items selected for a user or audience.
Baseline	A simple method used as a reference when evaluating more advanced models.

1.2 Business Scenario

A new movie-streaming service does not yet know enough about each visitor to personalize recommendations. The service needs a reliable list of widely liked movies for its home page. A movie should not rank first merely because one person gave it five stars. The method must therefore consider both average rating and number of ratings.

1.3 Questions to Answer

- Which movies have received the most ratings?
- Which movies have the highest average ratings?
- Why can average rating alone be misleading?
- How can rating volume and rating quality be combined?
- What limitations does one shared Top 10 list have?

1.4 Expected Data Flow

ratings.csv supplies user-item interactions. movies.csv supplies titles and genres. The notebook validates both files, summarizes the ratings, aggregates results by movie, joins the movie titles, ranks candidates, and displays the recommendations. The completed notebook and documentation are then published to GitHub.

Part 2 Set Up the Project

2.1 Choose an Environment

Use either Jupyter Notebook in VS Code or Anaconda, or Google Colab. The code is the same. When using Colab, upload the two CSV files into the session or mount Google Drive. Remember that temporary Colab files may disappear when the session ends.

2.2 Create the Project Structure

Create a folder named dsa4060-week1-recommender with the following structure:

```
dsa4060-week1-recommender/  
├── data/  
│   ├── movies.csv  
│   └── ratings.csv  
├── images/  
│   └── top10_recommendations.png  
├── notebooks/  
│   └── week1_popularity_recommender.ipynb  
└── .gitignore
```

```
└─ README.md
└─ requirements.txt
```

If the dataset is too large or its licence discourages redistribution, omit the CSV files and explain in README.md how to download them. The MovieLens small files are manageable, but the repository should still retain dataset attribution.

2.3 Download the Dataset

1. Open the GroupLens MovieLens datasets page at <https://grouplens.org/datasets/movielens/>.
2. Download MovieLens latest-small and extract the ZIP file.
3. Copy movies.csv and ratings.csv into the data folder.
4. Open the dataset README and note the rating scale, fields, and permitted use.
5. Do not rename columns inside the CSV files.

2.4 Install or Import Packages

In a local environment, install the required packages from the terminal:

```
python -m pip install pandas matplotlib jupyter
```

Create requirements.txt with:

```
pandas
matplotlib
jupyter
```

2.5 Create the Notebook

Create notebooks/week1_popularity_recommender.ipynb. Add a Markdown title cell containing the course, your name, student number, date, and the purpose of the practical. Use a new Markdown heading for every major step so that the notebook reads like a short technical report.

Part 3 Load and Inspect the Data

3.1 Import Libraries

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option('display.max_columns', None)
```

3.2 Define File Paths

The notebook is inside notebooks, so .. moves to the project root. If you use Colab, replace these paths with the location where you uploaded the files.

```
PROJECT_ROOT = Path('.')
DATA_DIR = PROJECT_ROOT / 'data'
IMAGES_DIR = PROJECT_ROOT / 'images'
IMAGES_DIR.mkdir(exist_ok=True)

MOVIES_FILE = DATA_DIR / 'movies.csv'
RATINGS_FILE = DATA_DIR / 'ratings.csv'
```

3.3 Load the CSV Files

```
movies = pd.read_csv(MOVIES_FILE)
ratings = pd.read_csv(RATINGS_FILE)

print('Movies shape:', movies.shape)
print('Ratings shape:', ratings.shape)
```

3.4 Preview and Inspect

```
display(movies.head())
display(ratings.head())

print(movies.info())
print(ratings.info())
```

Expected key columns are movieId, title, and genres in movies; and userId, movieId, rating, and timestamp in ratings. Exact row counts can change if GroupLens updates the latest-small dataset, so record the counts produced by your downloaded version.

3.5 Validate the Data

```
print('Missing values in movies:')
print(movies.isna().sum())

print('Missing values in ratings:')
print(ratings.isna().sum())

print('Duplicate movie rows:', movies.duplicated().sum())
print('Duplicate rating rows:', ratings.duplicated().sum())
print('Rating range:', ratings['rating'].min(), 'to', ratings['rating'].max())
print('Unknown movie IDs:', (~ratings['movieId'].isin(movies['movieId'])).sum())
```

Write two or three sentences interpreting the results. Do not remove rows simply because a duplicate count is nonzero. First decide which fields should uniquely identify a record. For example, repeated ratings by the same user and movie may represent data history or duplication depending on the dataset definition.

3.6 Summarize the Interaction Data

```
n_users = ratings['userId'].nunique()
n_movies_rated = ratings['movieId'].nunique()
n_interactions = len(ratings)

print('Unique users:', n_users)
print('Movies with at least one rating:', n_movies_rated)
print('Total rating interactions:', n_interactions)
print(ratings['rating'].describe())
```

In a Markdown cell, explain what each number represents. Distinguish the number of movies in movies.csv from the number of movies that have at least one rating.

Part 4 Explore User Item Interactions

4.1 Ratings per User and per Movie

```
ratings_per_user = ratings.groupby('userId').size()
ratings_per_movie = ratings.groupby('movieId').size()

print('Ratings per user:')
print(ratings_per_user.describe())
print()
print('Ratings per movie:')
print(ratings_per_movie.describe())
```

4.2 Examine the Rating Distribution

```
rating_distribution = ratings['rating'].value_counts().sort_index()
print(rating_distribution)

ax = rating_distribution.plot(kind='bar', color='#1F4E78', figsize=(8, 4))
ax.set_title('Distribution of Movie Ratings')
ax.set_xlabel('Rating')
ax.set_ylabel('Number of Ratings')
plt.tight_layout()
plt.show()
```

Interpret the chart. Which rating occurs most often? Are low ratings rare? State only what the chart establishes; do not assume the reason for the distribution.

4.3 Calculate Interaction Matrix Sparsity

A complete user-item matrix would contain one cell for every possible user-movie pair. Most cells are normally empty because each user rates only a small subset of movies.

```
possible_interactions = n_users * n_movies_rated
observed_interactions = n_interactions
sparsity = 1 - (observed_interactions / possible_interactions)

print(f'Possible interactions: {possible_interactions:,}')
print(f'Observed interactions: {observed_interactions:,}')
print(f'Sparsity: {sparsity:.2%}')
```

Explain what the calculated percentage means. High sparsity is one reason recommender systems need methods that can infer preferences from limited observations. This week's popularity baseline does not attempt to personalize empty user-item cells.

Part 5 Build a Popularity Recommender

5.1 Aggregate Ratings by Movie

```
movie_stats = (
    ratings.groupby('movieId')
    .agg(average_rating=('rating', 'mean'),
         rating_count=('rating', 'count'))
    .reset_index()
)

movie_stats = movie_stats.merge(
    movies[['movieId', 'title', 'genres']],
    on='movieId',
    how='left',
    validate='one_to_one'
)

display(movie_stats.head())
```

The merge adds readable titles to the aggregated statistics. `validate="one_to_one"` makes the code fail if either side contains unexpected duplicate movie IDs, which is safer than silently multiplying rows.

5.2 Most Rated Movies

```
most_rated = (
    movie_stats.sort_values(
        ['rating_count', 'average_rating'],
        ascending=[False, False]
    )
    .head(10)
)

display(most_rated[['title', 'rating_count', 'average_rating']])
```

This list represents visibility or engagement, not necessarily quality. A widely rated movie can have a moderate average rating.

5.3 Highest Average Ratings

```
highest_average = (
    movie_stats.sort_values(
        ['average_rating', 'rating_count'],
        ascending=[False, False]
    )
    .head(10)
)
```

```
display(highest_average[['title', 'rating_count', 'average_rating']])
```

Inspect `rating_count`. Movies with very few ratings may dominate this list. This is why average rating alone is a weak popularity baseline.

5.4 Minimum Ratings Baseline

Require a movie to have at least 50 ratings before ranking it by average rating. The threshold is a teaching choice, not a universal rule. Later, compare it with a threshold derived from the dataset.

```
MIN_RATINGS = 50

popular_movies = (
    movie_stats[movie_stats['rating_count'] >= MIN_RATINGS]
    .sort_values(
        ['average_rating', 'rating_count'],
        ascending=[False, False]
    )
    .head(10)
)

display(popular_movies[['title', 'genres', 'rating_count', 'average_rating']])
```

5.5 Convert the Logic into a Function

```
def recommend_popular_movies(stats, min_ratings=50, top_n=10):
    candidates = stats[stats['rating_count'] >= min_ratings].copy()
    return (
        candidates.sort_values(
            ['average_rating', 'rating_count'],
            ascending=[False, False]
        )
        .head(top_n)
        [['movieId', 'title', 'genres', 'rating_count', 'average_rating']]
    )

recommendations = recommend_popular_movies(movie_stats, 50, 10)
display(recommendations)
```

Part 6 Build a Weighted Rating Baseline

6.1 Why Use a Weighted Score

A minimum threshold discards all movies below the cutoff. A weighted rating instead pulls estimates for movies with few ratings toward the overall mean. Movies with many ratings are influenced more by their own average.

Use the formula: $\text{weighted score} = (v \div (v + m)) \times R + (m \div (v + m)) \times C$, where R is the movie average, v is its rating count, C is the overall mean rating, and m is the minimum evidence threshold.

6.2 Select the Threshold

```
C = ratings['rating'].mean()
m = movie_stats['rating_count'].quantile(0.90)

print(f'Overall mean rating C: {C:.3f}')
print(f'90th percentile rating count m: {m:.1f}')
```

Using the 90th percentile means that only movies in approximately the top 10% by rating count qualify. This is a transparent policy choice. Test a different percentile and explain how the candidate pool changes.

6.3 Calculate Weighted Scores

```
qualified = movie_stats[movie_stats['rating_count'] >= m].copy()

v = qualified['rating_count']
R = qualified['average_rating']
```

```

qualified['weighted_score'] = (
    (v / (v + m)) * R +
    (m / (v + m)) * C
)

weighted_recommendations = (
    qualified.sort_values(
        ['weighted_score', 'rating_count'],
        ascending=[False, False]
    )
    .head(10)
)

display(weighted_recommendations[[
    'title', 'genres', 'rating_count',
    'average_rating', 'weighted_score'
]])

```

6.4 Compare the Two Lists

```

comparison = popular_movies[['title', 'rating_count', 'average_rating']].copy()
comparison['method'] = 'Minimum 50 ratings'

weighted_view = weighted_recommendations[[
    'title', 'rating_count', 'average_rating'
]].copy()
weighted_view['method'] = 'Weighted score'

display(pd.concat([comparison, weighted_view], ignore_index=True))

```

Write a short comparison. Which titles appear in both lists? Which method is more conservative about small rating counts? Neither method is personalized, and neither guarantees diversity or novelty.

Part 7 Visualize and Test the Output

7.1 Create a Top 10 Chart

```

plot_data = weighted_recommendations.sort_values('weighted_score')

ax = plot_data.plot(
    x='title',
    y='weighted_score',
    kind='barh',
    figsize=(10, 6),
    color='#1F4E78',
    legend=False
)
ax.set_title('Top 10 Movies by Weighted Rating')
ax.set_xlabel('Weighted score')
ax.set_ylabel('Movie')
plt.tight_layout()
plt.savefig(IMAGES_DIR / 'top10_recommendations.png', dpi=150, bbox_inches='tight')
plt.show()

```

7.2 Perform Basic Tests

```

assert len(recommendations) <= 10
assert recommendations['movieId'].is_unique
assert recommendations['rating_count'].ge(50).all()
assert recommendations['average_rating'].between(0.5, 5.0).all()
assert recommendations['title'].notna().all()

assert len(weighted_recommendations) <= 10
assert weighted_recommendations['weighted_score'].notna().all()

print('All checks passed.')

```

Tests do not prove that the recommender is useful to every user. They check structural expectations and obvious calculation problems. Human review is still needed to assess the list and its limitations.

7.3 Try Different Parameters

```
display(recommend_popular_movies(movie_stats, min_ratings=20, top_n=5))
display(recommend_popular_movies(movie_stats, min_ratings=100, top_n=5))
```

Compare the results. A low threshold admits more items but can make rankings unstable. A high threshold favours widely rated movies but may exclude less-known high-quality items.

7.4 Optional Genre Filter

```
def recommend_by_genre(stats, genre, min_ratings=30, top_n=10):
    mask = stats['genres'].str.contains(genre, case=False, na=False)
    return recommend_popular_movies(stats[mask], min_ratings, top_n)

display(recommend_by_genre(movie_stats, 'Comedy', 30, 10))
```

This is still non-personalized because every person requesting Comedy receives the same list. It is a simple contextual filter, not a learned user preference model.

Part 8 Interpret the Results

8.1 Questions for the Notebook

1. Which movie received the most ratings, and what was its average rating?
2. Which movie ranked first after applying the 50-rating threshold?
3. Which movie ranked first using the weighted score?
4. How many movies qualified under the 90th-percentile threshold?
5. What changed when the minimum rating threshold was increased?
6. Why is the resulting list not personalized?
7. What bias may result from recommending only already-popular movies?
8. How could the system handle a new movie with no ratings?

8.2 Required Discussion

Add a Markdown section titled Findings and Limitations. Write at least 250 words explaining the main dataset observations, the difference between the ranking methods, the role of rating count, and at least three limitations. Suitable limitations include popularity bias, lack of personalization, cold start for new movies, changing preferences over time, and the fact that a rating does not reveal why a user liked an item.

8.3 Recommended Improvements

- Use genres or textual features to build a content-based model.
- Use user-rating patterns to build collaborative filtering.
- Separate training and test data and evaluate recommendation quality.
- Add diversity, novelty, and coverage measures rather than optimizing only rating score.
- Consider time and recency because preferences and popularity can change.

Part 9 Prepare the GitHub Repository

9.1 Complete the README File

Create README.md in the project root. Use the following structure and replace placeholders with your own results:

```
# DSA 4060 Week 1 Popularity Recommender
```

```

## Student
- Name: Your Name
- Student number: Your Number

## Project overview
This project explores MovieLens user-item ratings and builds two
non-personalized movie recommendation baselines.

## Dataset
MovieLens latest-small from GroupLens. Include the source link,
files used, and date accessed.

## Methods
1. Rating-count and average-rating exploration
2. Minimum-rating popularity baseline
3. Weighted-rating baseline

## How to run
1. Clone the repository
2. Install packages: `pip install -r requirements.txt`
3. Place the required CSV files in `data/` if they are omitted
4. Open `notebooks/week1_popularity_recommender.ipynb`
5. Run all cells from top to bottom

## Key findings
Summarize your actual results.

## Limitations
Explain why the recommendations are not personalized and discuss bias.

## Repository structure
Describe the folders and important files.

## Screenshot
![[Top 10 recommendations](images/top10_recommendations.png)]

```

9.2 Create the Git Ignore File

```

.ipynb_checkpoints/
__pycache__/.
.venv/
venv/
.env
.DS_Store

```

Do not place a GitHub personal access token in the notebook, README, .env file, or screenshots. The .env entry prevents accidental tracking only if the file has not already been committed.

9.3 Clean the Notebook

1. Restart the notebook kernel or runtime.
2. Run all cells from the first cell to the last without manual intervention.
3. Resolve every error and remove unused exploratory cells.
4. Keep important outputs and charts visible.
5. Confirm that paths are relative and do not contain your private computer username.
6. Save the notebook after the final run.

Part 10 Upload the Project to GitHub

Option A Upload Using the GitHub Website

1. Sign in to GitHub and select New repository.
2. Name it dsa4060-week1-recommender and add a short description.

3. Choose Public unless the lecturer instructs you to use a private repository.
4. Do not initialize with a README if you already created README.md locally.
5. Create the repository, then choose Add file and Upload files.
6. Upload README.md, requirements.txt, .gitignore, the notebooks folder, the images folder, and permitted data files.
7. Use the commit message Complete Week 1 popularity recommender and commit the changes.
8. Open the notebook and README on GitHub to confirm that they render correctly.

Option B Upload Using Git Commands

Open a terminal in the project root and run the commands below. Replace YOUR-USERNAME with your actual GitHub username.

```
git init
git add .
git status
git commit -m "Complete Week 1 popularity recommender"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/dsa4060-week1-recommender.git
git push -u origin main
```

Before git commit, inspect git status. Confirm that the notebook, README, requirements file, image, and permitted dataset files are included, and that secrets or unrelated files are absent. If GitHub requests authentication, use the supported sign-in flow or a personal access token; never paste the token into project files.

10.1 Verify the Online Repository

Check	Expected evidence
Repository opens	The submitted URL opens the correct project.
README renders	Purpose, dataset, methods, findings, limitations, and run instructions are visible.
Notebook renders	Markdown, code, tables, and charts are readable on GitHub.
Project structure	Notebook, image, requirements, and documentation are in clear folders.
Reproducibility	A reviewer can obtain the data and follow the instructions.
Security	No passwords, tokens, or personal data are committed.

Part 11 Submission Requirements

Required Deliverables

1. The GitHub repository URL.
2. A completed and executed Jupyter notebook.
3. README.md with dataset attribution, setup instructions, findings, and limitations.
4. requirements.txt and .gitignore.
5. The saved Top 10 recommendation chart in the images folder.
6. One screenshot showing the repository home page and one showing the notebook output.

Suggested Marking Guide 30 Marks

Criterion	Marks	Evidence
Project setup and organization	3	Clear folders, notebook metadata, requirements, and gitignore.

Criterion	Marks	Evidence
Data loading and validation	5	Correct loading, inspection, missing/duplicate checks, and interpretation.
Exploratory analysis	5	User-item summaries, rating distribution, and sparsity calculation.
Popularity recommender	5	Correct aggregation, thresholding, function, and Top N output.
Weighted recommender and comparison	5	Correct score, ranked output, and method comparison.
Visualization and testing	3	Saved chart and relevant assertions.
Interpretation and limitations	2	Evidence-based findings and meaningful limitations.
GitHub documentation and submission	2	Readable repository, README, attribution, and working URL.
Total	30	

Final Student Checklist

- All notebook cells run from top to bottom without errors.
- Every result is explained in Markdown, not only displayed.
- The recommender returns no more than the requested number of unique movies.
- The chart is saved and displayed in README.md.
- Dataset source and access date are acknowledged.
- The README gives complete instructions for reproducing the work.
- The repository contains no passwords, access tokens, or private data, and its link opens in a private browser window.

Reflection Questions

1. What did the popularity baseline do well?
2. Which users or movies might be disadvantaged by this approach?
3. What additional data would be needed to personalize recommendations?
4. Which result would you use as the home-page baseline and why?
5. What will you change in the next version of the recommender?

End of practical. Retain this repository because later recommender models can be added and compared with the Week 1 baseline.